

Feedback neural networks are neural networks with the ability to provide bottom-up and top-down design feedback to their input or previous layers, based on their outputs or subsequent layers. This is notably used in large language models, specifically in reasoning language models (RLM). This process is designed to mimic self-assessment and internal deliberation, aiming to minimize errors (like hallucinations) and increase interpretability. This reflection is a form of "test-time compute", where additional computational resources are used during inference.

Introduction

Traditional neural networks process inputs in a feedforward manner, generating outputs in a single pass. However, their limitations in handling complex tasks, and especially compositional ones, have led to the development of methods that simulate internal deliberation. Techniques such as chain-of-thought prompting encourage models to generate intermediate reasoning steps, thereby improving their performance in such tasks.

The feedback can take place either after a full network pass and decoding to tokens, or continuously in latent space (the last layer can be fed back to the first layer). In LLMs, special tokens can mark the beginning and end of reflection before producing a final response (e.g., <thinking>).

This internal process of "thinking" about the steps leading to an answer is designed to be analogous to human metacognition or "thinking about thinking". It helps AI systems approach tasks that require multi-step reasoning, planning, and logical thought.

Techniques

Increasing the length of the Chain-of-Thought reasoning process, by passing the output of the model back to its input and doing multiple network passes, increases inference-time scaling. Reinforcement learning frameworks have also been used to steer the Chain-of-Thought. One

example is Group Relative Policy Optimization (GRPO), used in DeepSeek-R1, a variant of policy gradient methods that eliminates the need for a separate "critic" model by normalizing rewards within a group of generated outputs, reducing computational cost. Simple techniques like "budget forcing" (forcing the model to continue generating reasoning steps) have also proven effective in improving performance.

Types of reflection

Post-hoc reflection

Analyzes and critiques an initial output separately, often involving prompting the model to identify errors or suggest improvements after generating a response. The Reflexion framework follows this approach.

Iterative reflection

Revises earlier parts of a response dynamically during generation. Self-monitoring mechanisms allow the model to adjust reasoning as it progresses. Methods like Tree-of-Thoughts exemplify this, enabling backtracking and alternative exploration.

Intrinsic reflection

Integrates self-monitoring directly into the model architecture rather than relying solely on external prompts, enabling models with inherent awareness of their reasoning limitations and uncertainties. This has been used by Google DeepMind in a technique called Self-Correction via Reinforcement Learning (SCoRe) which rewards the model for improving its responses.

Process reward models and limitations

Early research explored PRMs to provide feedback on each reasoning step, unlike traditional reinforcement learning which rewards only the final outcome. However, PRMs have faced

challenges, including computational cost and reward hacking. DeepSeek-R1's developers found them to be not beneficial.

See also

Reflective programming

Reservoir computing

== References ==